Sheep user guide

Sheep runs AI models on your own computer. You ask in plain words and Sheep does the work: it answers, writes documents, workbooks, decks and videos, searches the web when a question is about now, and shows everything it made on a Work Canvas.

Open and start

1. Open **Sheep** from the desktop shortcut.
2. The first time, choose how to start: add a model from Hugging Face, use model files already on this computer, or start at once with Nemotron on Nebius (with a Nebius key). Your last model loads by itself after that. The lamp at the top shows its state: *Starting* → *Checking the model* → *Ready*.
3. Type in the message box and press Enter.

The card above the message box shows the checks as they run ("Checking app tasks · 3 of 14"). You can chat while they run. The **Guide** in the sidebar shows the basics again any time.

The tour

Click **Take the tour** at the top left. Sheep's own sheep, with a shepherd's crook and a clipboard, walks to every part of the app and points at each piece as it names it. Each stop says what the part is, how to use it, what it gains in speed or quality, and its limits, for this computer (its own graphics cards, processor and heat) and for the model plugged in now. **Next** moves on; **Esc** leaves.

Click the sheep to ask instead: it puts its crook behind its back and a search box opens. Type what you want to know (models, GGUF, video, ads, heat, export, Nebius...), pick a topic, and it walks there and explains it. Click it again to carry on the tour.

Two modes

Mode	What it means
Sheep	Everything runs on this computer. Nothing is sent anywhere, except web searches, which you can switch off.
Sheep + Nebius	NVIDIA Nemotron on Nebius plans long jobs and checks the local model's work; Sheep runs the tools and makes the files on this computer. Every request sent out is listed in **Settings › Advanced › Sent out**, with who answered it.

Making things

Ask for what you want, for example "*Make a workbook called budget.ssheet with Rent 800, Food 250 and a Total row*". Sheep uses its own apps:

App	Files	What it does
Sheep Words	.sdoc (imports and exports .docx)	documents with headings, lists, tables, pictures
Sheep Sheets	.ssheet (imports and exports .xlsx, .csv)	workbooks with formulas, charts, sorting
Sheep Slides	.sdeck (imports and exports .pptx)	decks with layouts, themes, pictures
Sheep Studio	.sstudio, renders .mp4	videos and motion graphics: text, pictures, clips, shapes, music, templates
Sheep Text	.txt, .md, code	notes, text and code

Every change can be undone (the arrows at the top right). A file made in chat shows as a card with **Open**, its Office copy (**.docx**, **.xlsx**, **.pptx**) and **Canvas**; typing just `pptx`, `docx` or `xlsx` after a job saves the Office copy too.

Videos

- **A video from words**: "*make a 2-second video of waves at sunset*". The chat model first writes the scene for the video model (subject, setting, light, camera), and the reply shows the scene it used. Local video models make short clips (1 to 6 seconds, 512 pixels high) on a 4 GB graphics card.
- **Animate a photo**: attach or select a picture and ask Sheep to *animate this photo*.
- **A photo montage**: ask for a *slideshow* or *montage* of the selected photos.
- **An ad from photos**: select up to 12 photos and ask for *an ad for my bakery from these photos*. Each photo fills the frame with a slow pan and zoom, the photos cross-fade, a headline and tagline come in over them (written by the chat model from your request), and an end card closes. Name a sound file (*with music.mp3*) for music and ask for a voice-over to have it spoken. Say *vertical* for 9:16 or *square* for 1:1.
- **Watch a video**: attach a video, or ask *what happens in clip.mp4*. Sheep takes the key frames where the picture changes, describes each one (with a picture-reading model), and listens to the sound: what is said and when, the silences, the loud moments and the beat. The summary is saved beside the video and Studio can use it ("cut the silent parts").

In **Sheep Studio** every picture or clip can be cropped (*Crop the picture*), set to fill its box or show all of it, faded in and out by the seconds you choose, and split in two at the playhead.

The Work Canvas

Everything a chat makes appears as a **block**: a document with a preview, a sheet, a deck, a video or sound you can play, a picture, code, notes and web sources. The Work Canvas opens beside the chat when a job makes something; the **Canvas** button at the top shows it at any time.

- **Open** a document, sheet or deck in its app.
- **Run** a code block (Python or JavaScript) in a safe, separate process; the output appears under it.
- **Copy** or **save** code and notes as files.
- **Pin** a block to keep it on top, and **drag** blocks to reorder them.

The same blocks also show in the chat, under the answer that made them.

Answers from the web

A question about now (news, prices, scores, releases) is searched on the web. Sheep reads the best pages and answers with numbered sources [1], [2], shown as cards under the answer. Every search and page read is listed in **Settings › Advanced › Sent out**; web search is switched on or off in **Settings › Basics**.

Models

Settings › Models is the one place to choose models. It lists one row for each job (chat, checker, pictures, video, photo guidance, speech to text, speech out) with its status, **Ready** or **Missing**, and **Choose another…** to pick any Hugging Face model for that job. Sheep checks a model fits this computer before anything downloads, and gives a new picture or speech model a test run before it is used.

- Pictures and video: any Stable Diffusion 1.5 model (the many community styles) becomes the base for both. Bigger families such as SDXL are refused with the reason (they need about 8 GB of graphics memory).
- Speech to text: any faster-whisper model, from *tiny* (fast) to *large-v3* (most accurate).

When a request needs a model you do not have, the reply says what is missing and shows a button to download the suggested one or **Choose another…**, then **Try again**.

Below the jobs are your chat models as cards: the name, the size, where it runs, and **Use**, **Publish** and **Remove**. To add a chat model, use **Add any model**:

1. Give a Hugging Face ID (such as `Qwen/Qwen2.5-1.5B-Instruct`), a folder of model files, or a `.gguf` file, and press **Check**. Nothing downloads yet.
2. Sheep says what the model is and shows the sizes it can build for this computer: from Q2 (the smallest) to Q8, or the weights as published. Each card shows the size, the quality, the speed here, whether it fits, and the build time. The recommended one is chosen already.
3. Press **Build**. The steps show as they happen: found, converted if it had to be, built, then checked.

A GGUF, GPTQ, AWQ, EXL2 or MLX page is traced to the original model it was made from, and Sheep builds from that. A model that exists only as a GGUF or GPTQ copy is converted first. A model Sheep's engine does

not know is fitted to it, or runs with its maker's own code (compatibility mode, slower but correct). If something is wrong (no internet, a misspelt name, a full disk), Sheep says what to do.

Settings › Activity › What Sheep did for you shows every step for the model in use: added, read, converted, built, placed, answers checked, app tasks checked, and any repair.

To share a model you built, sign in to Hugging Face with a token of type **Write** and press **Publish** on its card. It is published privately first, with the original's licence.

The checks

A new model first answers a short exam; Sheep is usable as soon as it passes (**Model ready**). Then check questions with known answers (**Answers checked**) and real jobs in Sheep's apps, each checked by opening what the model made (**Apps checked**). These run in the background, step aside whenever you ask something, and wait only when the graphics card reaches 95 °C or the computer runs on battery. A model that already passed on this computer, unchanged, skips the checks it passed. **Settings › Activity** shows every check, row by row.

Every reply is watched for faults: empty replies, loops, leaked markers, broken JSON or code. If a model misbehaves, Sheep repairs it in the background, and in Sheep + Nebius mode Nemotron answers meanwhile.

You can add your own, smaller **Checker model** (Settings › Advanced › Checker). It sits an exam and replaces the current Checker only if it does better. Its opinions are advice shown with a reply; they never rewrite or block what Sheep says.

Settings

Settings › Basics holds the everyday choices:

- **How Sheep runs**: *Fastest* uses every graphics card that helps; *Balanced* (the default) is fast and leaves the computer free for other work; *Cool and quiet* keeps 2 GB of graphics memory free and uses one card.
- **Where answers come from**: only this computer, or this computer with NVIDIA Nemotron on Nebius. The same switch sits under the chat box.
- **Search the web**, **the agent may** (ask first, edit freely, plan only), your workspace folder, and light or dark.

Settings › Models chooses the model for each job (see Models). **Settings › Accounts** holds the Nebius, Tavily and Hugging Face keys. **Settings › Activity** shows what Sheep did and how the model runs. Everything else is in **Settings › Advanced**: changing one of those makes "How Sheep runs" read *Custom*.

This computer

Settings › Advanced › This computer shows what Sheep found: the processor, any NVIDIA graphics card, and any AMD or Intel graphics. Choose where text models run:

Choice	Meaning
Auto	Sheep decides: the graphics card when the model fits, card and processor together when part fits, else the processor
A graphics card	the whole model on that card
Card + processor	the first part on the card, the rest on the processor, for models larger than the card
Processor	the processor only

A choice this computer cannot run is greyed out, with the reason on hover. If a graphics card fails, the reply finishes on the processor and Settings says why.

The **heat guard** lets everything run at full speed and pauses only when the graphics card reaches 95 °C (the processor sensor 103 °C), so the computer does not shut down in the middle of your work. On battery, background checks wait for the charger. After a new model is added, Sheep times it on each device here (speed trial) and tries a few engine settings (engine tune), keeping one only if the answer stays word for word the same.

Privacy

In Sheep mode your conversations, files and models stay on this computer. Web searches send only the search words and the page addresses read, and each one is logged. In Sheep + Nebius mode, each cloud request is logged with its reason and the provider currently configured for this setup.

Troubleshooting

Problem	What to do
The lamp stays on *Starting*	**Settings › Activity** shows the stage and any fault.
A reply is slow	**Settings › Activity** shows where the model runs and its speed. Close other programs that use the graphics card, or build a smaller size.
"Sheep could not reach Nebius"	Check the internet connection; in Sheep mode the local model answers with no connection.
"Nebius did not accept the key"	Paste the right key in **Settings › Accounts**.
A model was refused	The reason says what to do instead.
A picture, video or speech request says something is missing	Press the button in the reply (or open **Settings › Models**), then **Try again**.

Problem	What to do
Web answers are missing	Check that web search is on in **Settings › Basics**.

Licences

Sheep's own code is in this repository; see LICENSE and NOTICE. Models carry their own licences, shown in Settings › Models. Llama models are **Built with Llama** and keep "Llama" at the start of their name.

Sheep inference architecture families

The bottom of **Guide** has a dropdown of every Hugging Face `model_type` identifier currently marked `ok` by the inference build gate in `tooling/sml_archs.py`. It is a repository support list, not a scan of models installed on this computer. A family entry is not a guarantee that every checkpoint in that family has been independently validated.

The architecture roster has 140 entries: 91 accepted by the build gate, 17 with known engine gaps, 24 out of scope, and 8 unknown.

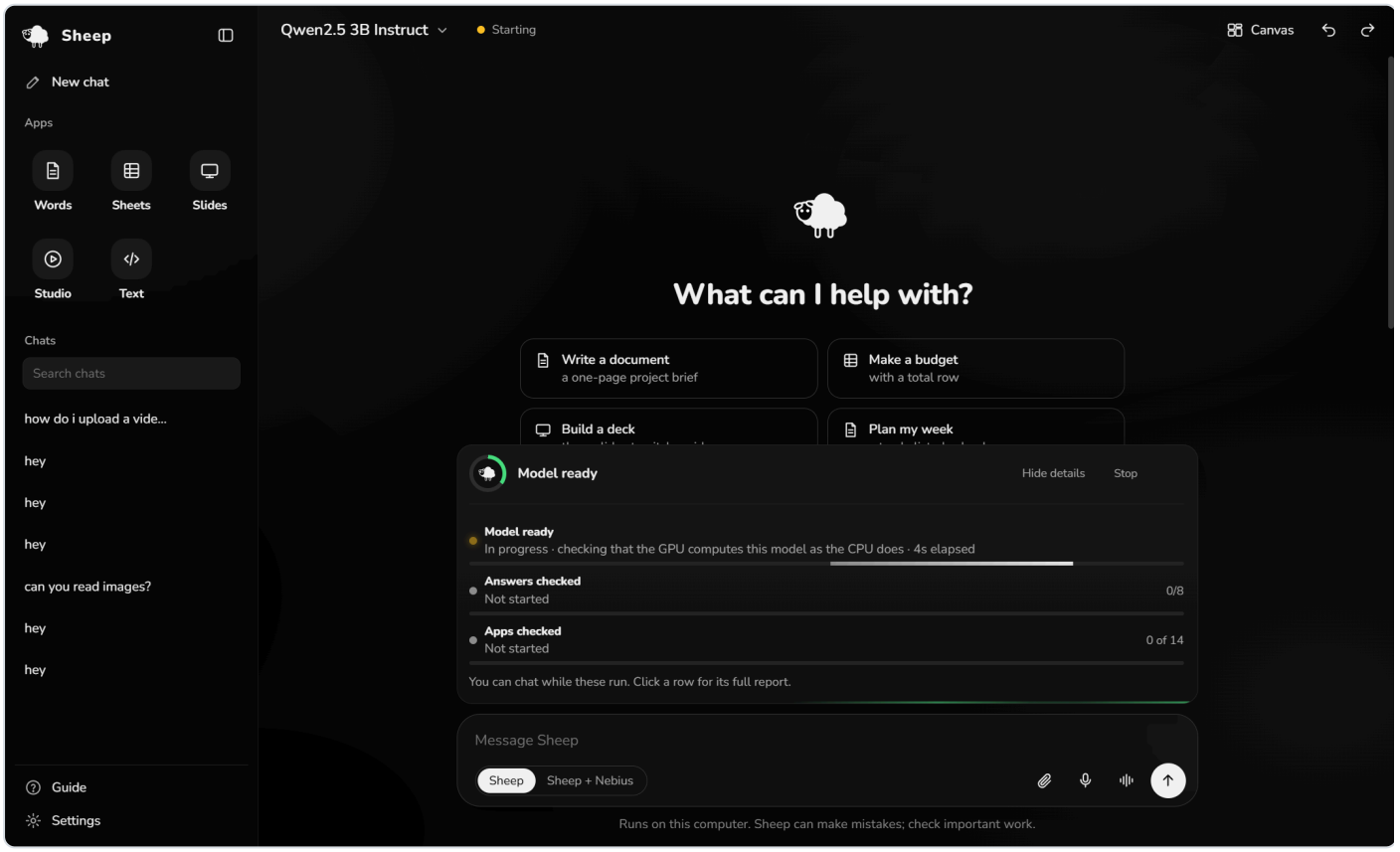
Checked end to end: 90 of 91 accepted families (`python tooling/sml_compat.py`). For each, the family's maths matches transformers -- a random-weight model of that family built from transformers' own code, in a small shape and in the shape of a big checkpoint (128-wide heads, one shared K/V head) -- and a real checkpoint of it passes every build check from its headers alone: tensor names and layouts, rotary scaling, tokenizer. Size does not change either check; the largest checkpoint read was deepseek-ai/DeepSeek-V3 (684.5B parameters). Family heads run: 91, 89 pass.

Accepted but not yet checked end to end: `openai_moe` (maths unproven).

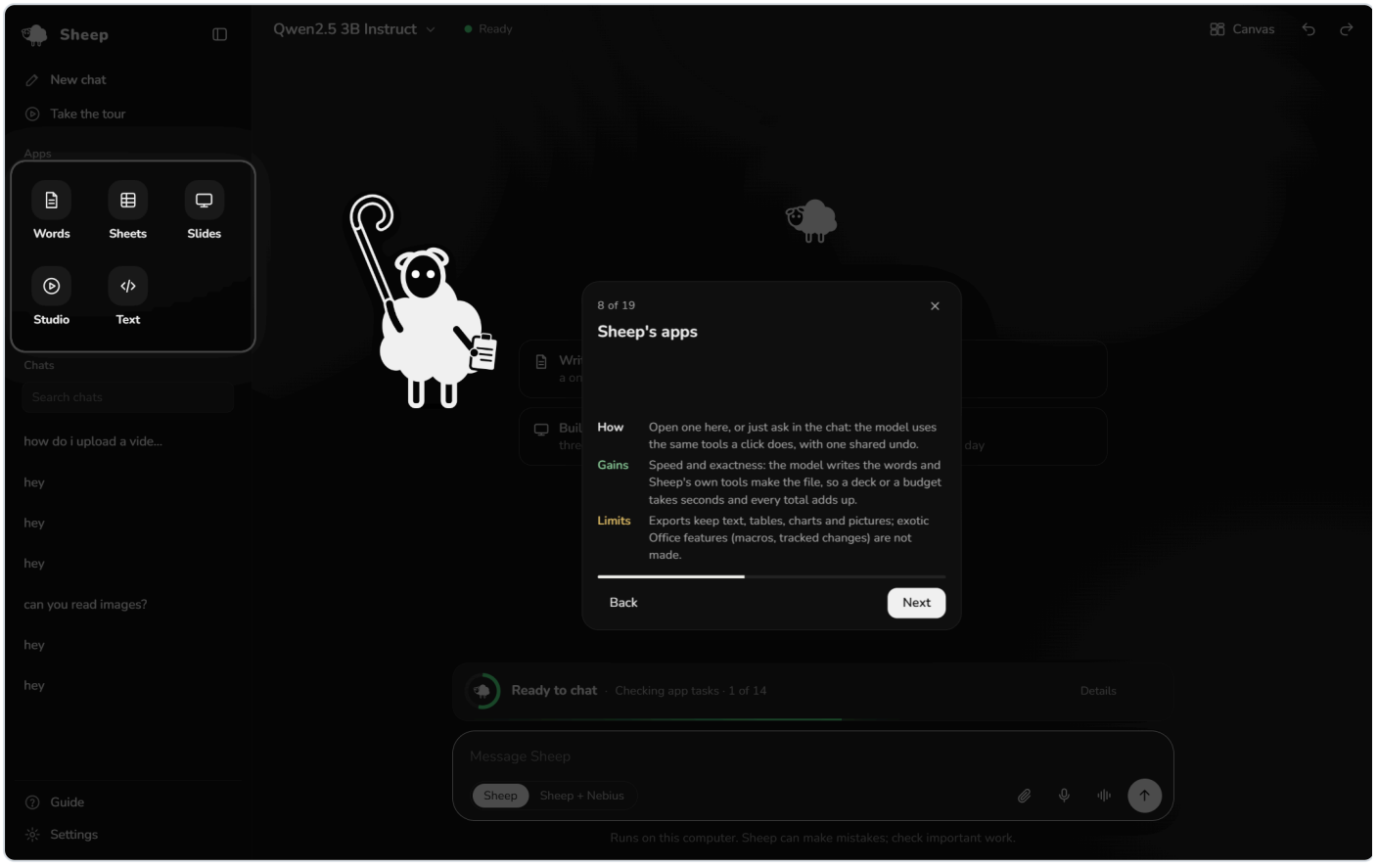
Real-checkpoint grade evidence exists for ten families: Llama, Qwen2, GPT-NeoX, Qwen3, Granite MoE, GPT-2, OLMo 2, BLOOM, SmolVLM, and GPT-Neo. Llama's current source is content-pinned but has no upstream revision; GPT-Neo's one-hour grade used a revision-pinned 125M checkpoint and matched the full-precision top token on 6 of 20 prompts. Eight older grades lack retained checkpoint pins. Only Llama, Qwen2, and GPT-NeoX have recorded GPU grades; the other seven are CPU-only. GPU support is narrower and the runtime refuses unsupported families.

Performance depends on model size and the device's RAM, CPU, GPU, and GPU memory. Choosing where models run in **Settings › Advanced › This computer** can change speed and whether a model fits. **Auto** is the recommended starting point; a family being listed does not guarantee that every checkpoint fits or runs on every device.

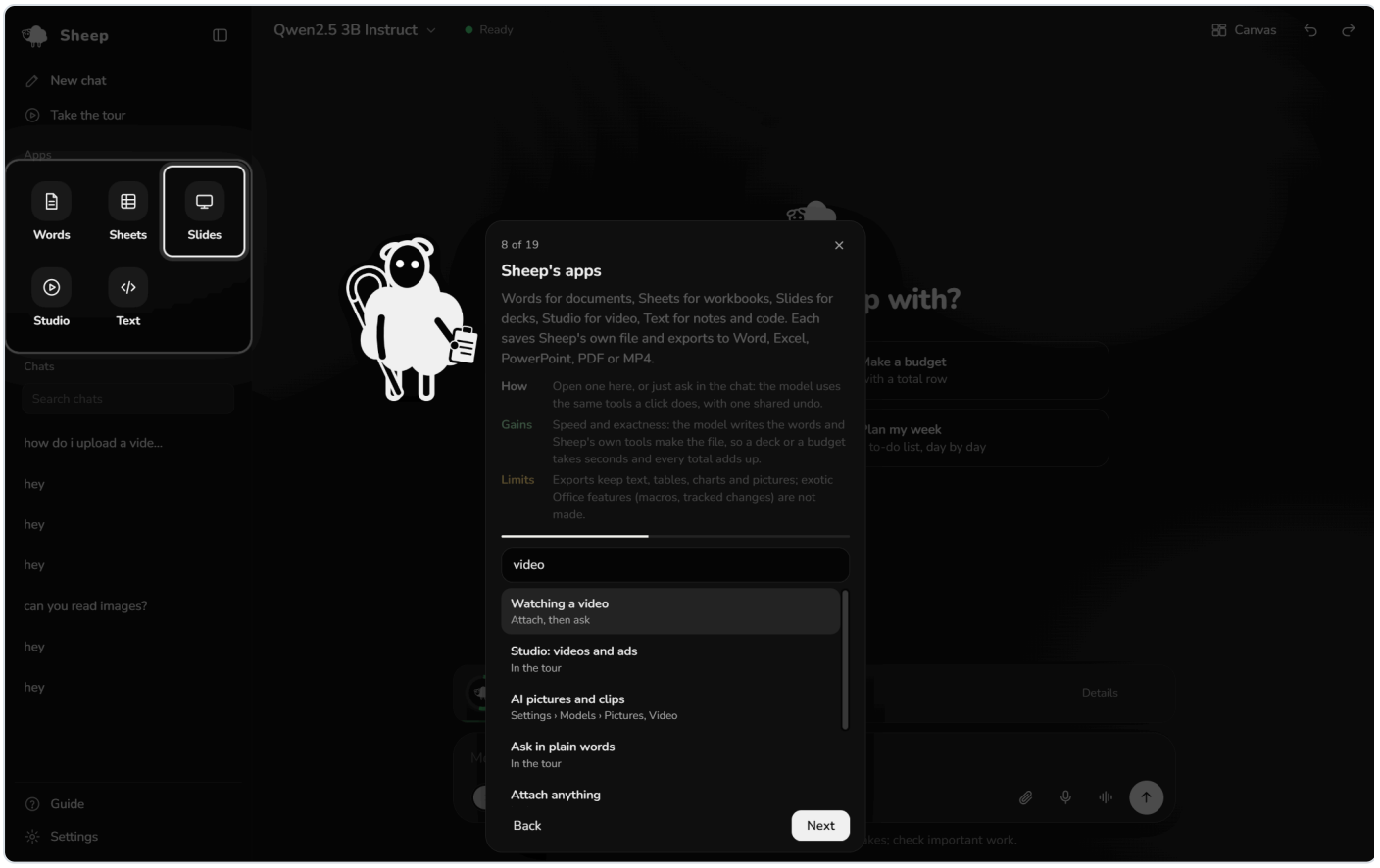
The screens



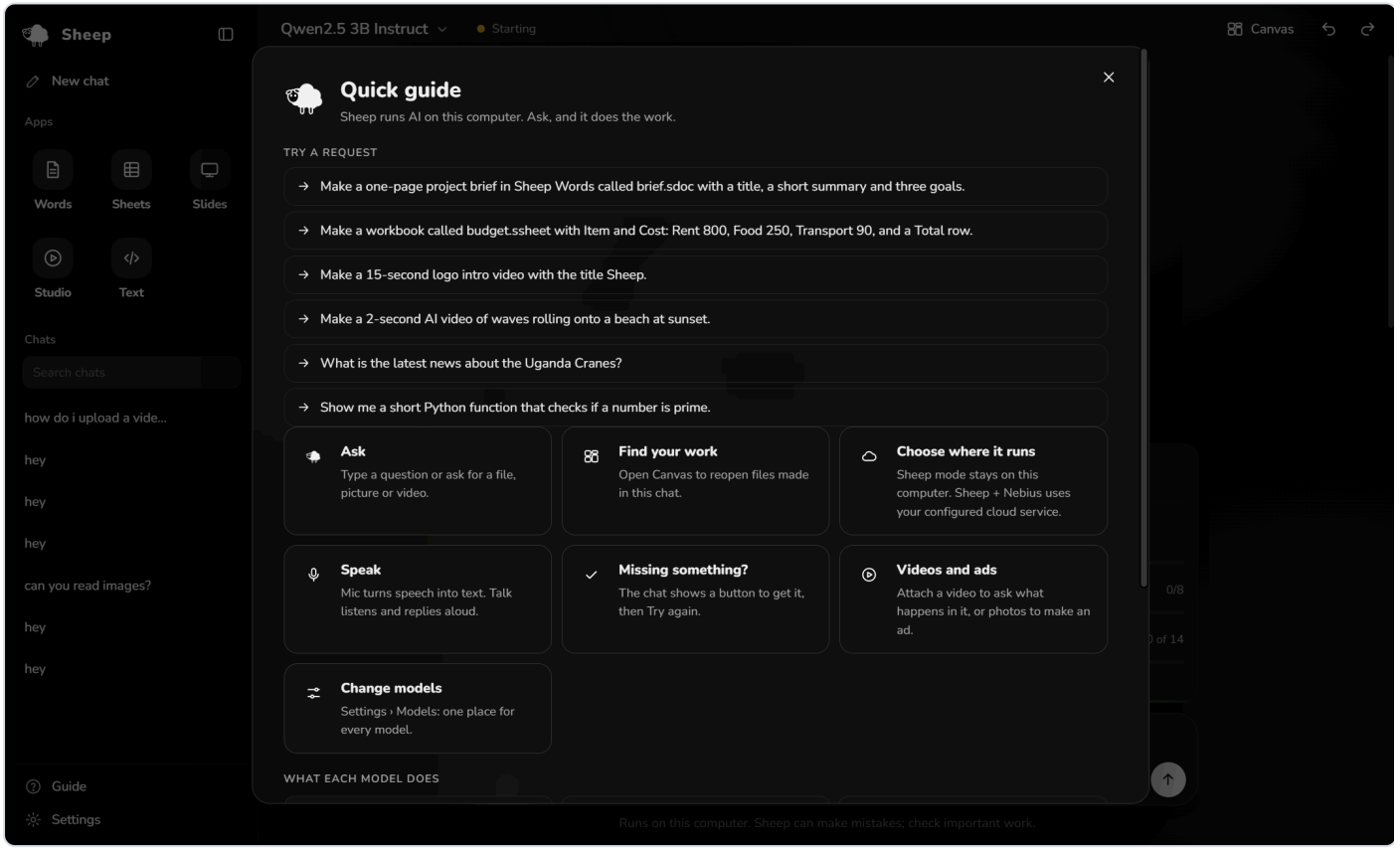
Chat, with the start-up card



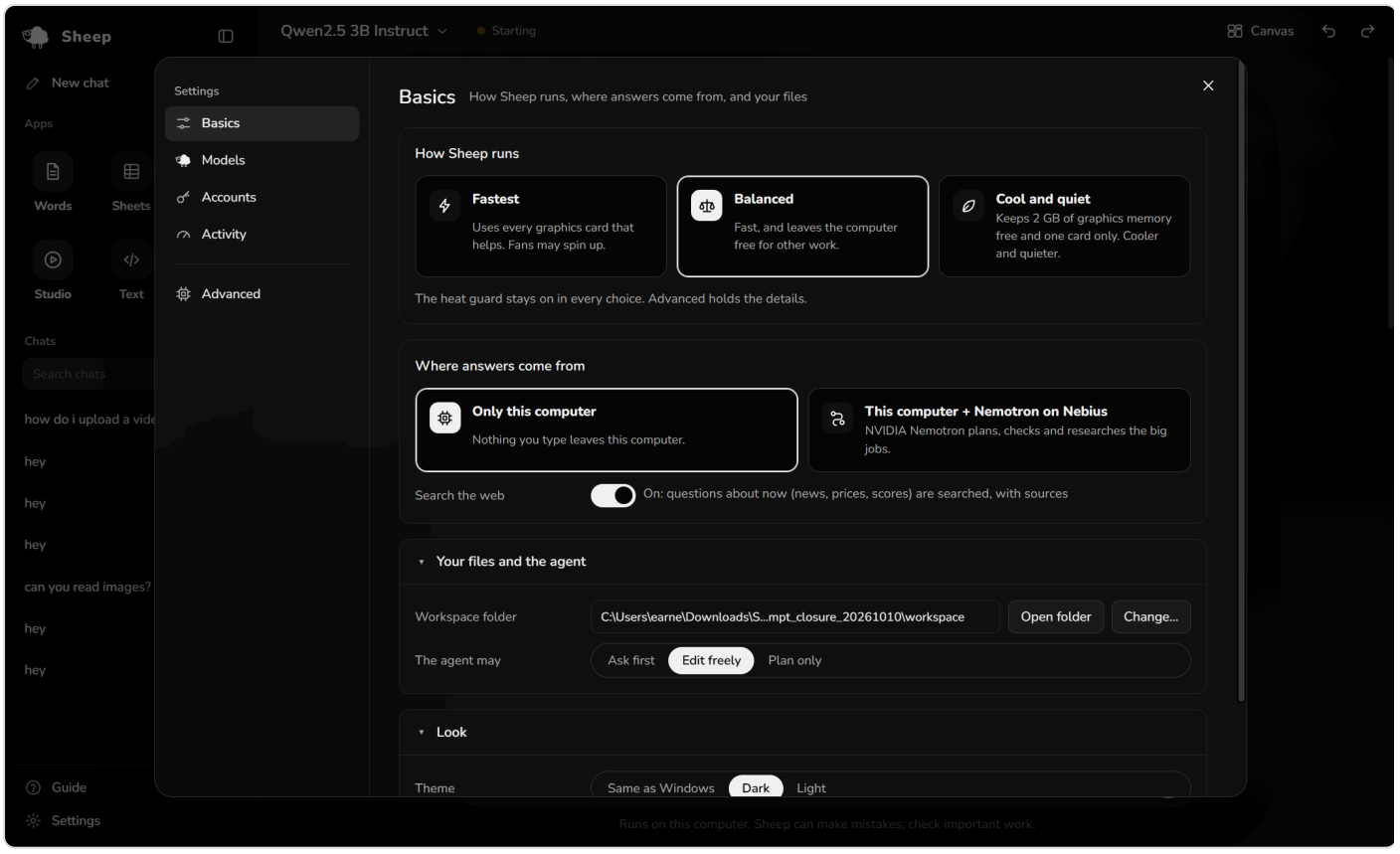
The tour



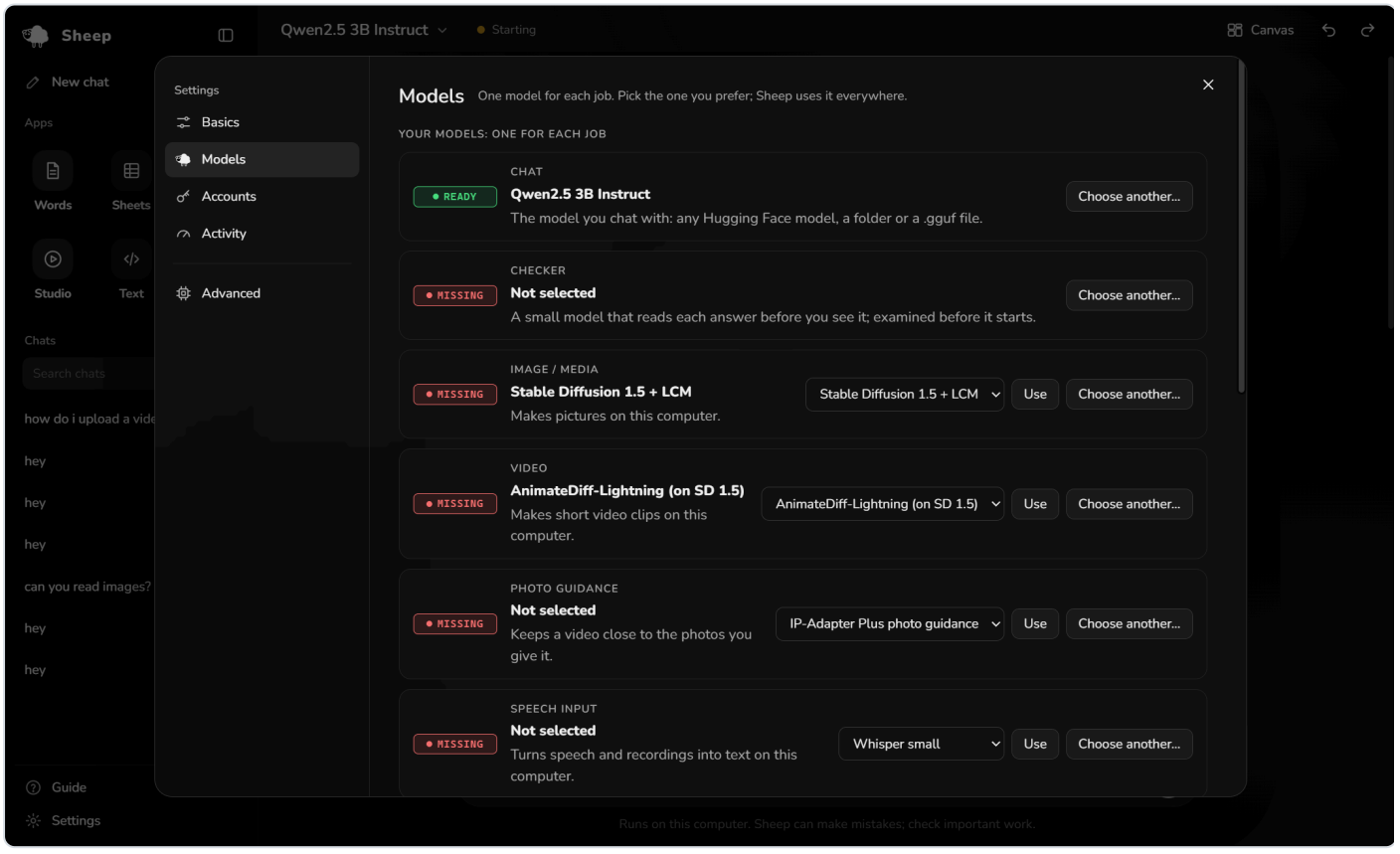
Asking the guide



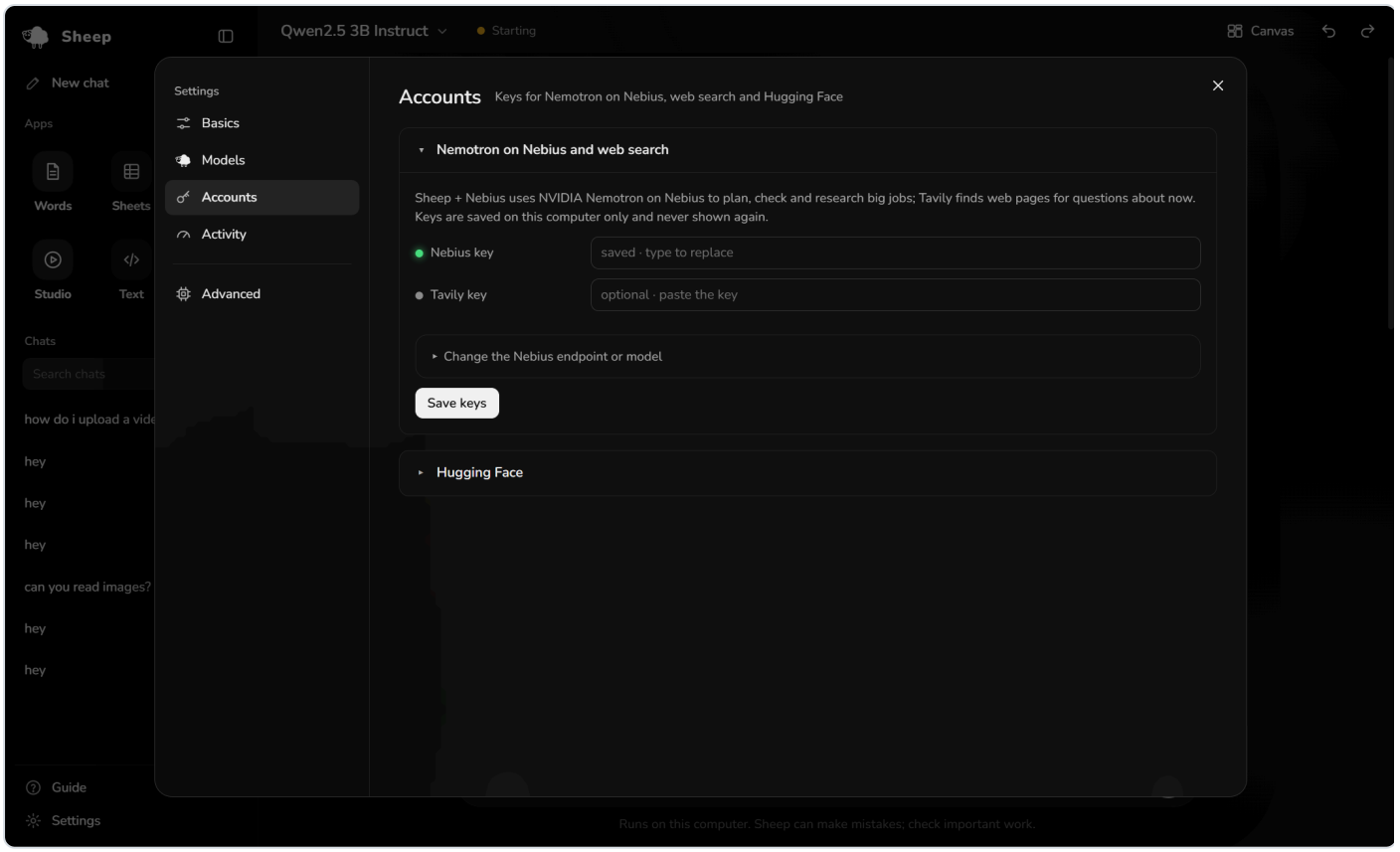
Quick guide



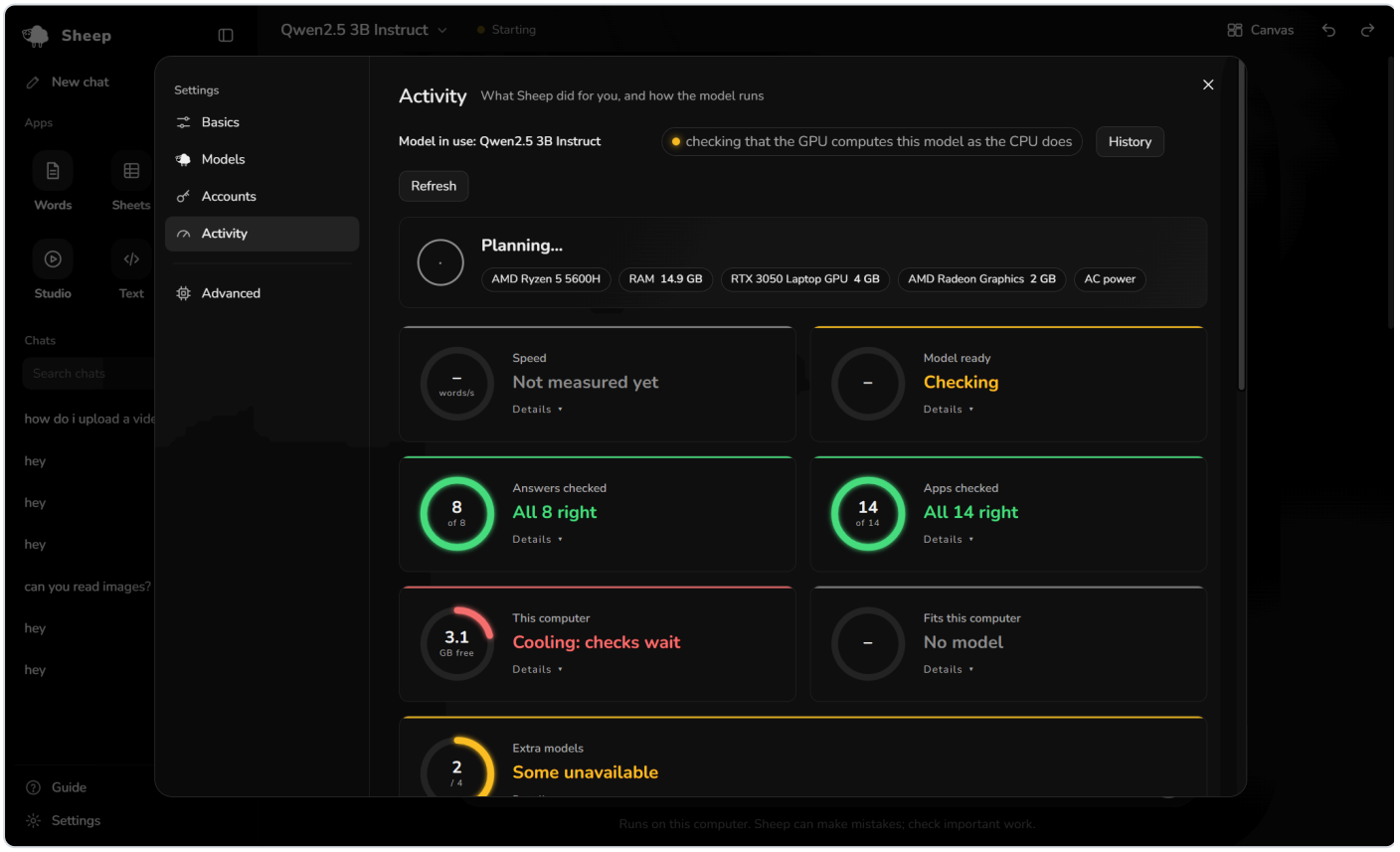
Settings › Basics



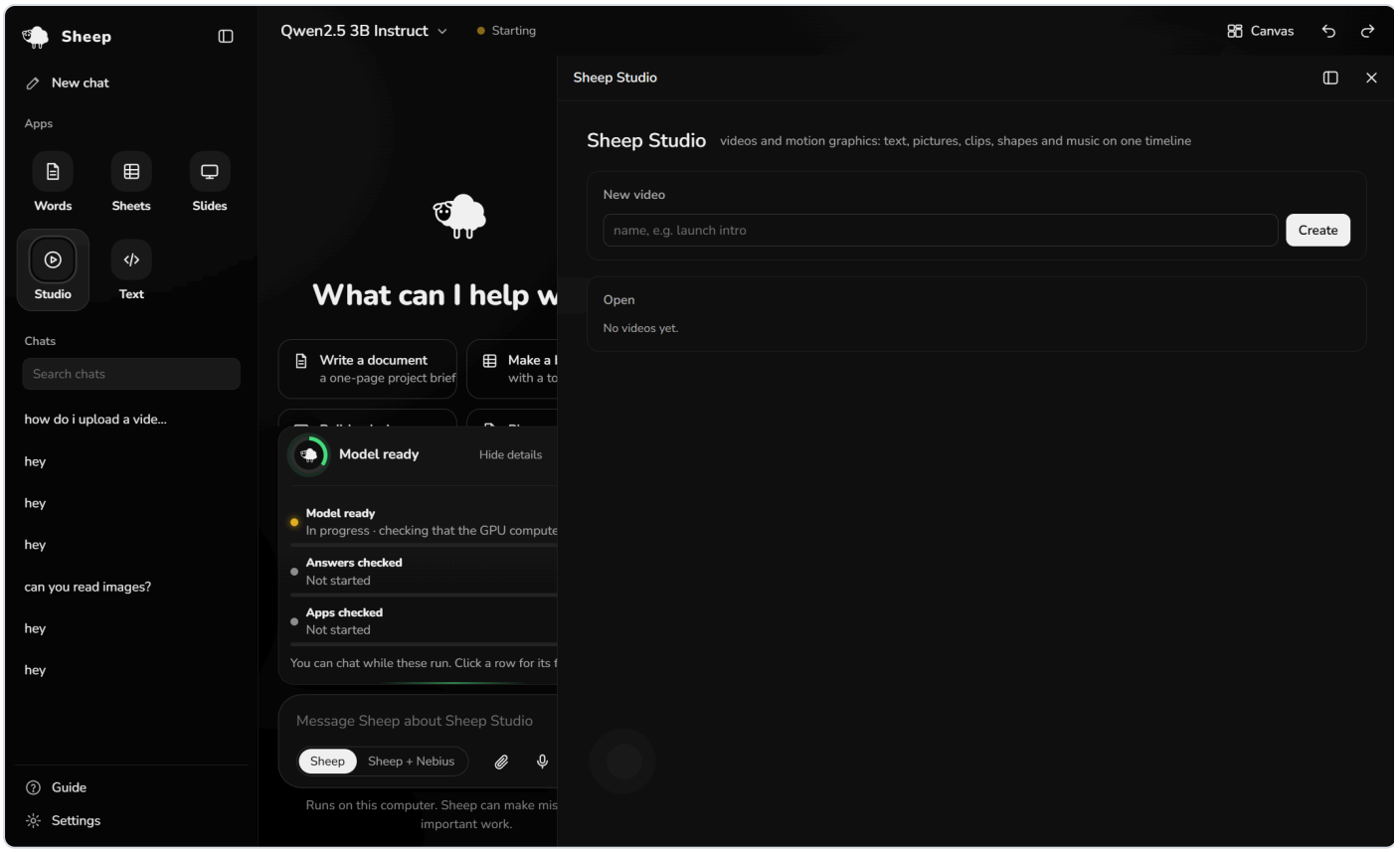
Settings › Models: one row per job



Settings › Accounts

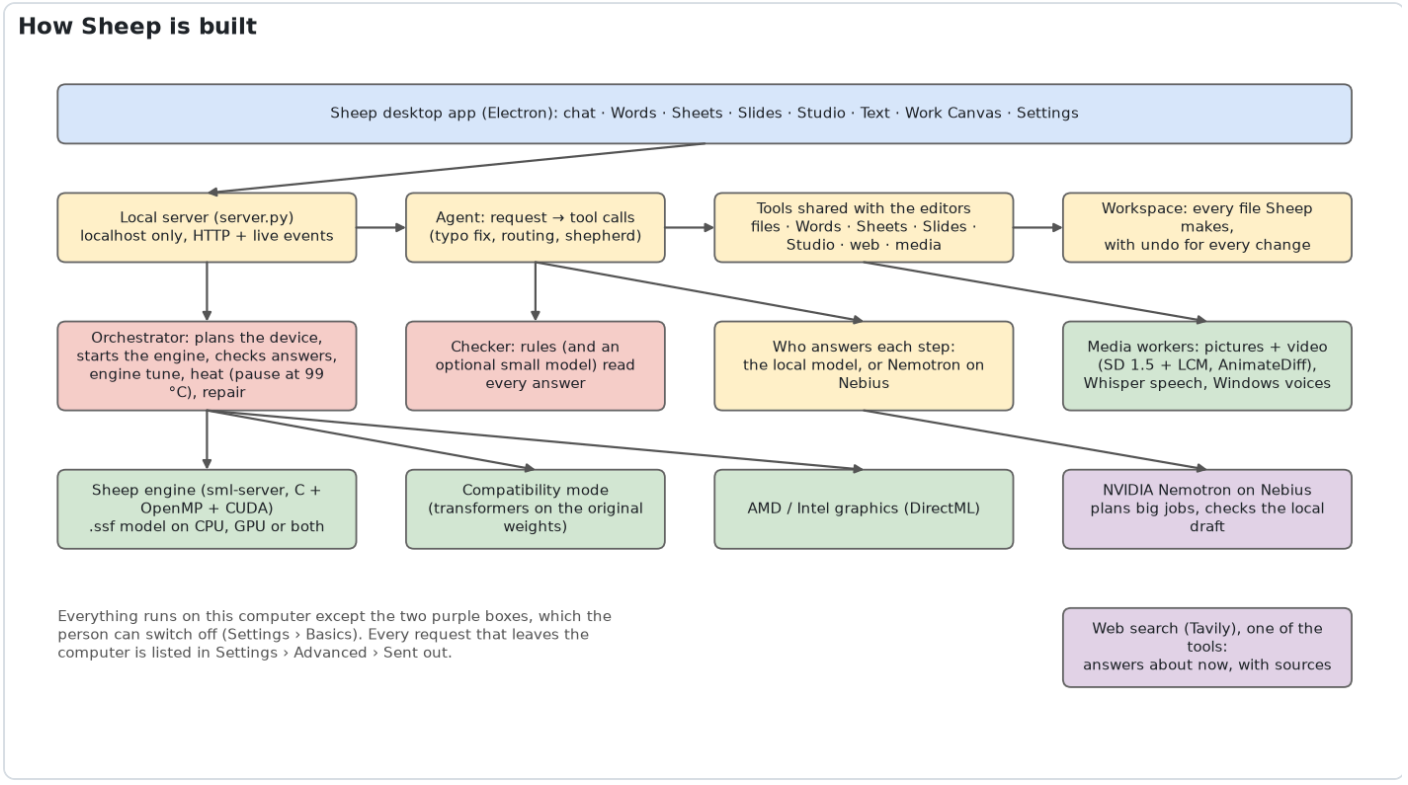


Settings › Activity

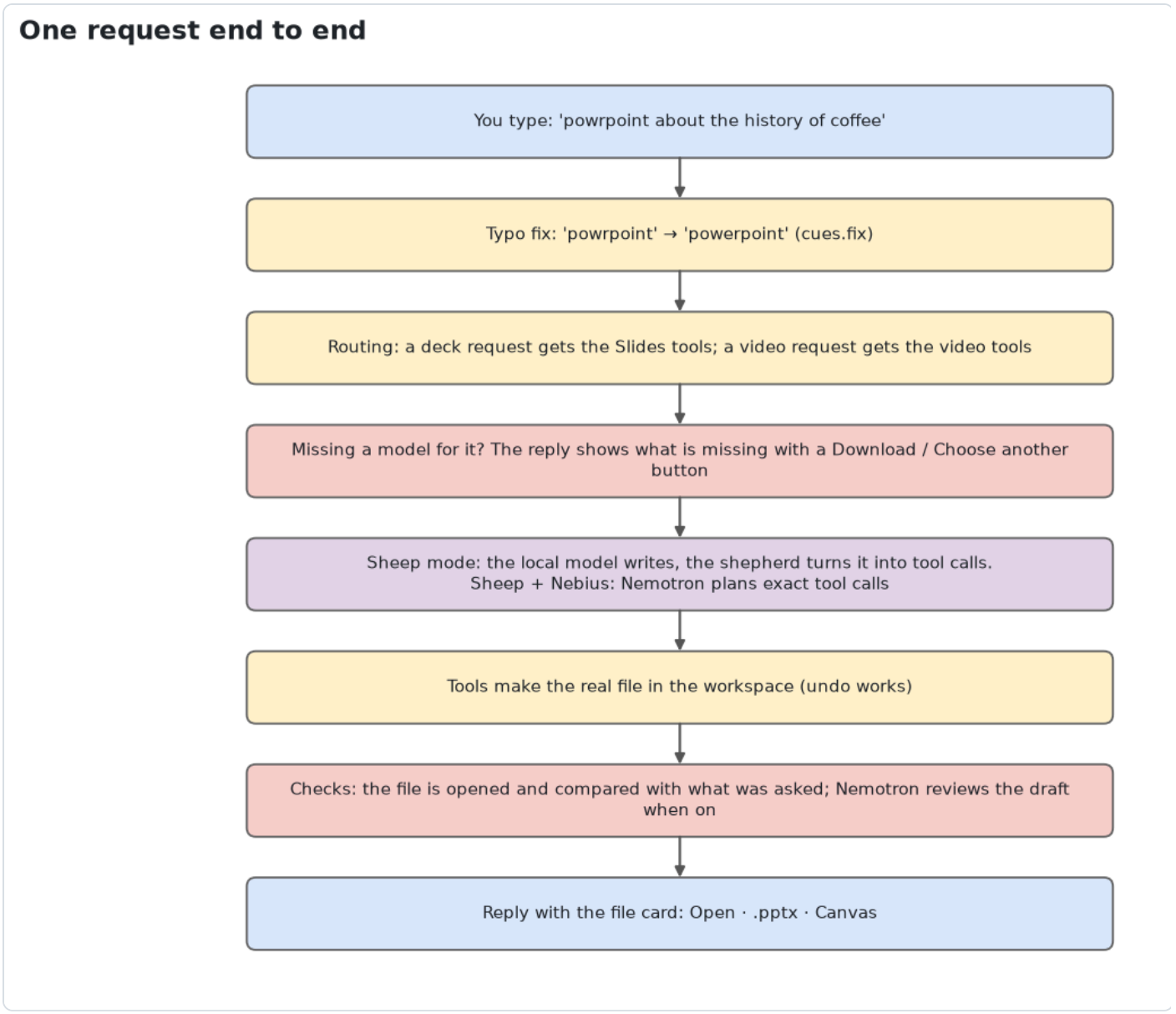


Sheep Studio

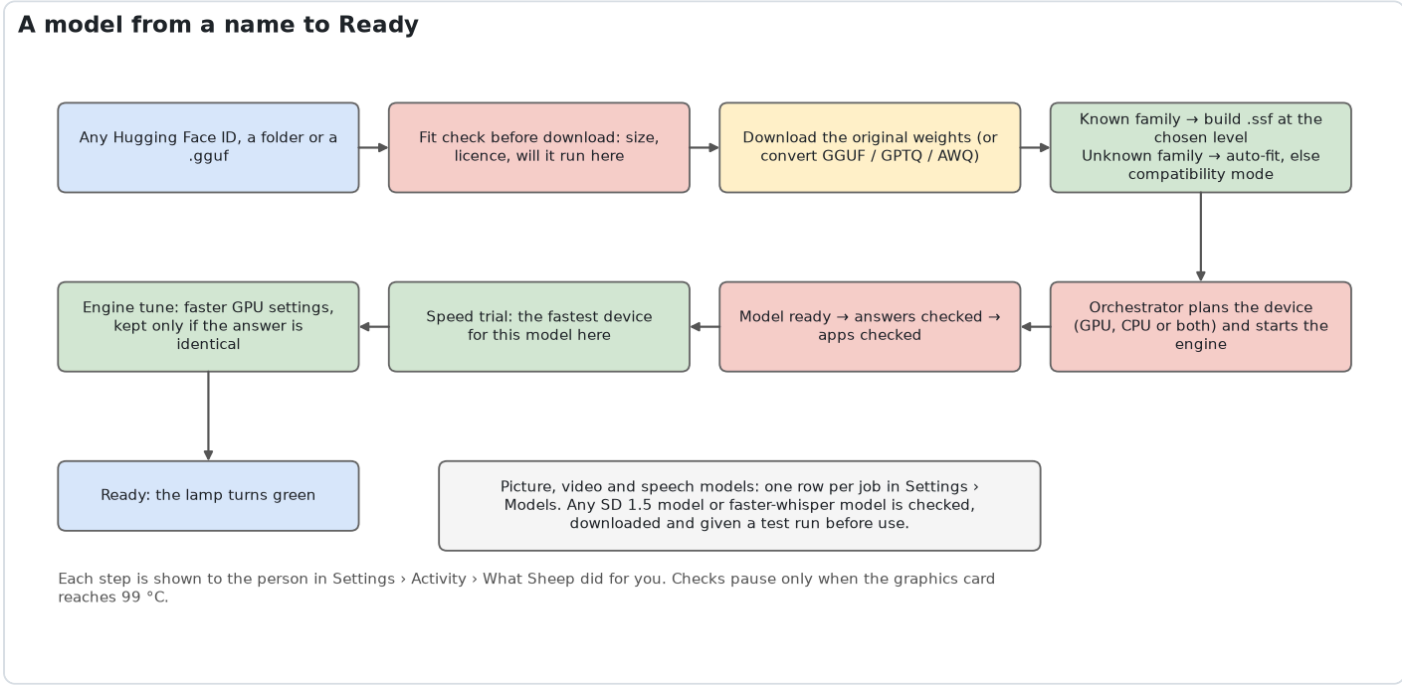
How it fits together, and how it measures up



How Sheep is built



One request, end to end

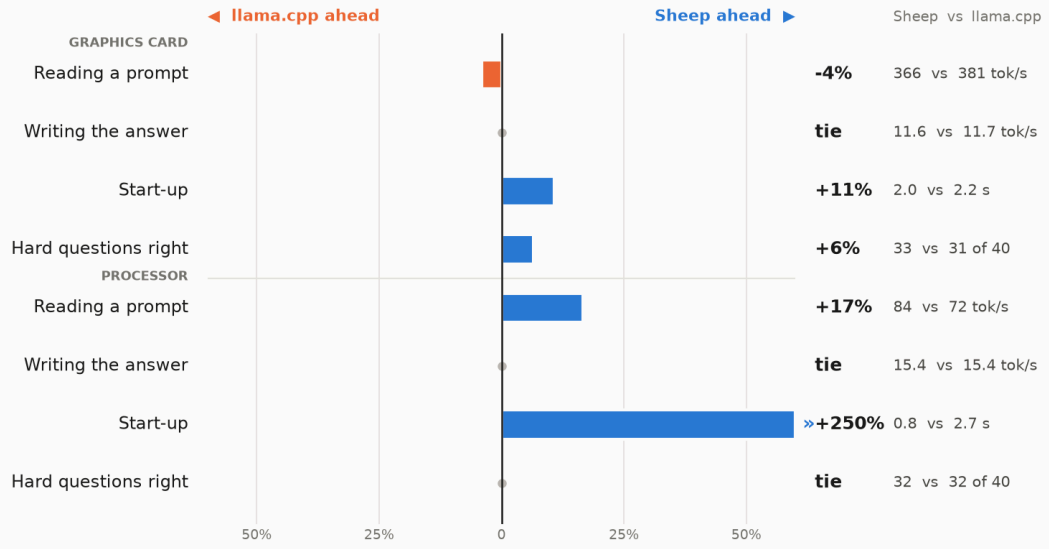


A model, from a name to Ready

Sheep against llama.cpp

Of 8 tests, Sheep is ahead in 4, behind in 1, tied in 3.

Ryzen 5 5600H · RTX 3050 Laptop 4 GB · on AC · Qwen2.5-3B at 4 bits on every engine. Speeds: median of the runs; questions: 40, at temperature 0.



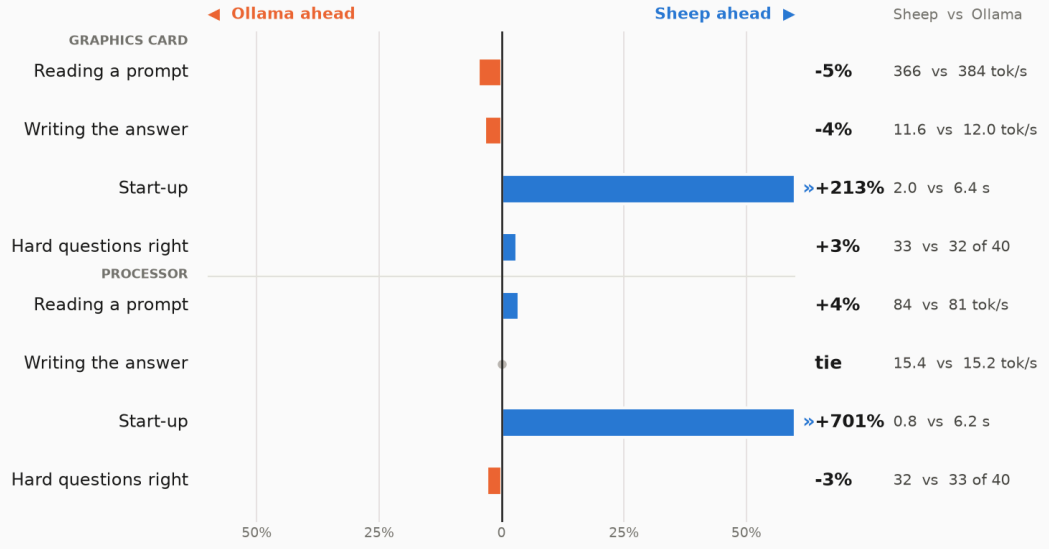
tooling/market_race.py · bar = Sheep's lead in % of the other's result · within 2% = tie · » = the lead runs past the scale

Sheep against llama.cpp

Sheep against Ollama

Of 8 tests, Sheep is ahead in 4, behind in 3, tied in 1.

Ryzen 5 5600H · RTX 3050 Laptop 4 GB · on AC · Qwen2.5-3B at 4 bits on every engine. Speeds: median of the runs; questions: 40, at temperature 0.



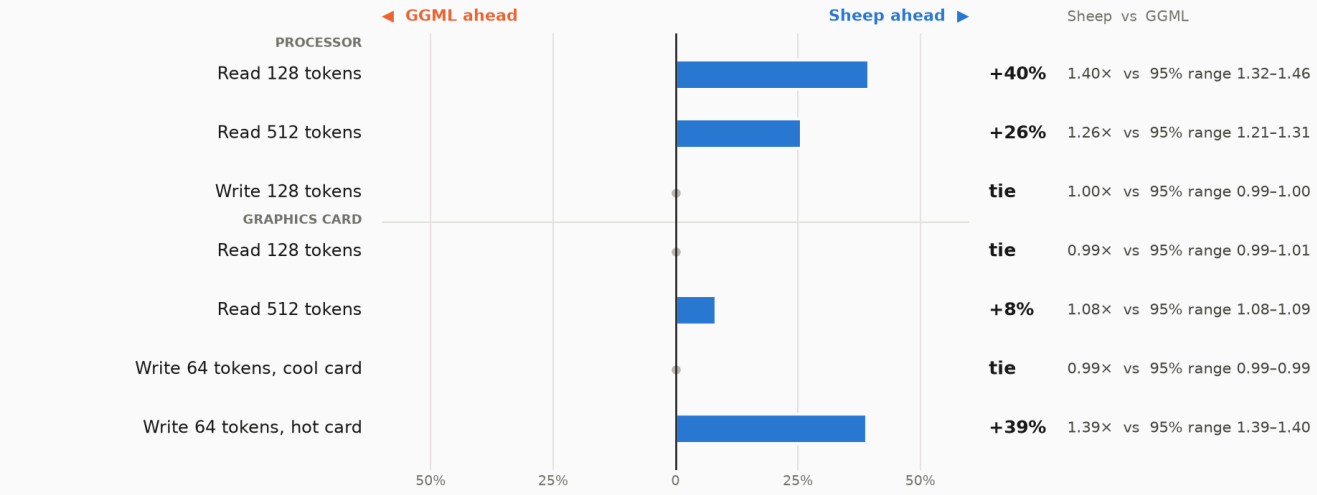
tooling/market_race.py · bar = Sheep's lead in % of the other's result · within 2% = tie · » = the lead runs past the scale

Sheep against Ollama

Engine against engine: Sheep's SML and GGML

Of 7 tests, Sheep is ahead in 4, behind in 0, tied in 3.

GGML is the engine inside llama.cpp and Ollama. Same laptop, same model, same 4-bit budget; 8 rounds in ABBA order each, with a bootstrap 95% range.



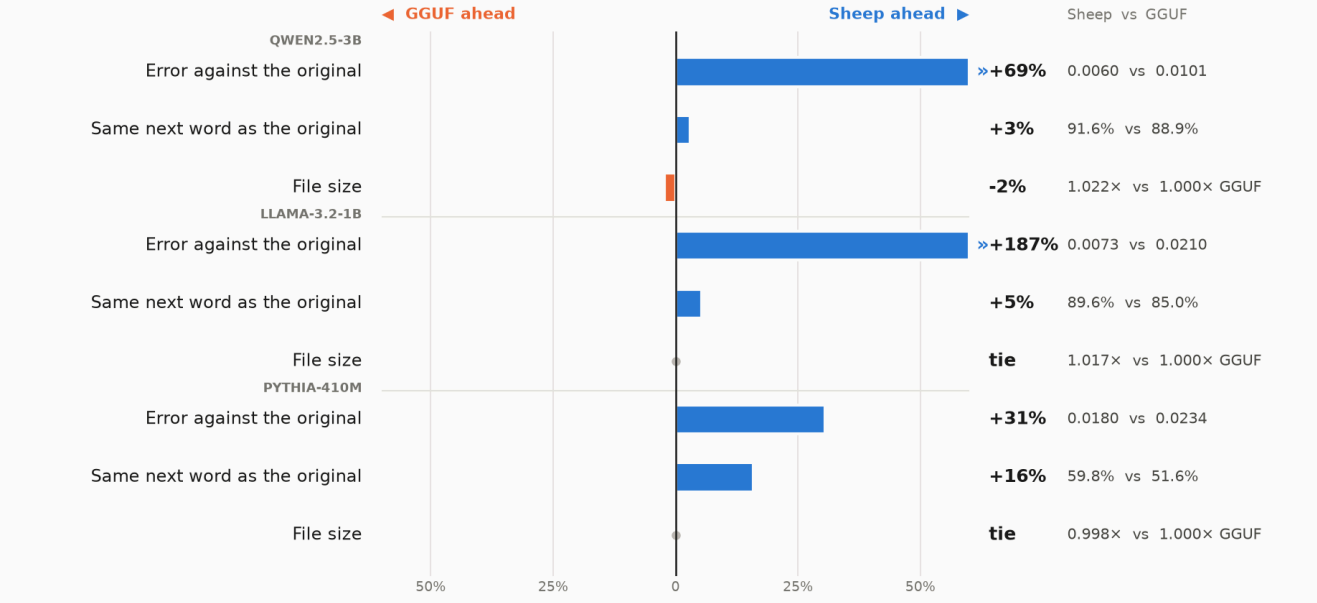
tooling/sml_cpu_race_ci.py, sml_gpu_race_ci.py · a range that crosses 1.00× is a tie

Engine against engine: SML and GGML

File against file: Sheep's SSF and GGUF Q4_K_M

Of 9 tests, Sheep is ahead in 6, behind in 1, tied in 2.

Same model, same 4-bit budget, scored against the full-precision model. Ollama loads the same GGUF, so this is SSF against both.

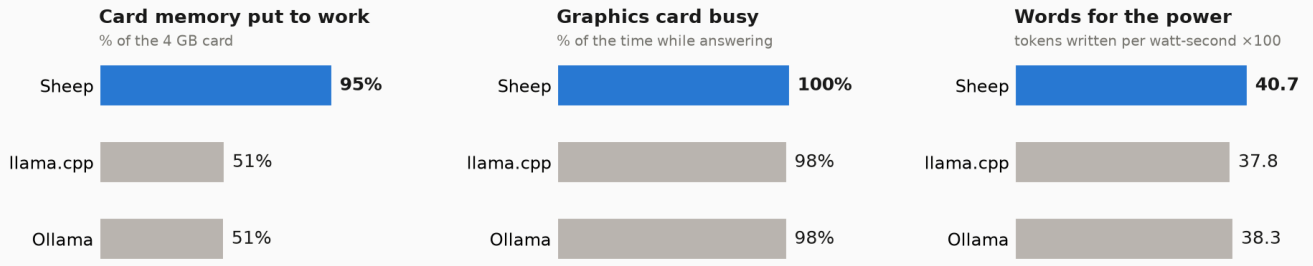


RESULTS.md, "Three specs raced" · every position of 256-512 held-out tokens scored

File against file: SSF and GGUF

How much of the machine each engine puts to work

On the graphics card, Sheep fills the card's spare memory with ready-made copies of the weights, keeps the card the busiest, and writes the most words per watt.



tooling/hw_race.jsonl (market_race.py: card memory counted from before the engine started; busy and power averaged over the runs).
Memory (RAM) on the card runs: Sheep 1.9 GB, llama.cpp 2.3 GB, Ollama 0.8 GB.

Hardware used for the same work

What else each one does

Beyond running a model. ✓ yes ◐ partly or with add-ons – no. Rows where the others are ahead are listed too.

Sheep alone has 10 of these 14. The others are ahead on 2.

	Sheep	llama.cpp	Ollama
Desktop app: chat window, no terminal	✓	◐	✓
Makes documents, workbooks and slide decks	✓	–	–
Makes pictures and video clips	✓	–	–
Video editor (Studio: crop, split, fades, ads)	✓	–	–
Reads attached videos, pictures, documents	✓	◐	◐
Speech in (mic) and spoken replies	✓	–	–
Web answers with sources	✓	–	◐
Its own 4-bit format and quantizer	✓	✓	–
Fits any model to this computer by itself	✓	◐	◐
Checks a model's answers before you use it	✓	–	–
Pauses at 99 °C instead of letting the laptop shut down	✓	–	–
Optional cloud help (NVIDIA Nemotron on Nebius)	✓	–	◐
Big ready-made model library to pull from	◐	◐	✓
Runs on Mac and Linux too	–	✓	✓

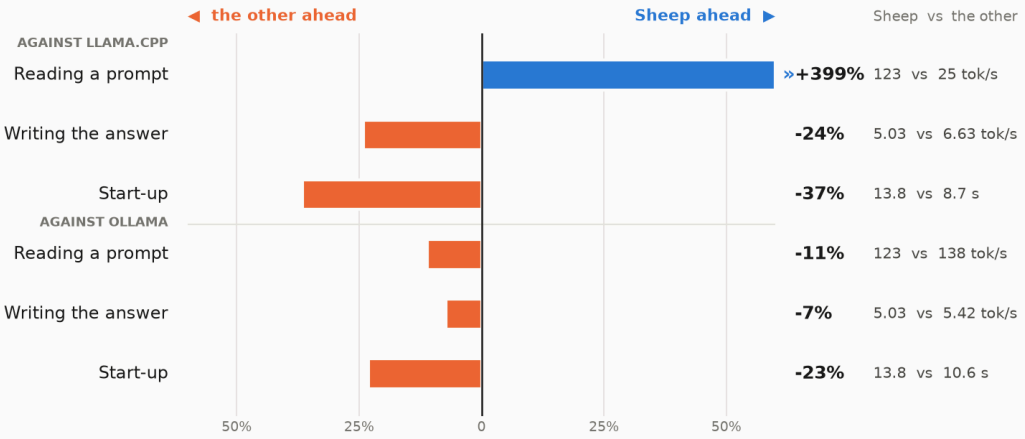
llama.cpp: llama-server has a basic web page; reads pictures with multimodal models. Ollama: desktop app and web search (account needed); cloud models through ollama.com.

What else each one does

A model too big for the card: Qwen2.5-7B on a 4 GB GPU

Of 6 tests, Sheep is ahead in 1, behind in 5.

4.7 GB at 4 bits; each engine splits it between the card and the processor by itself. Sheep reads every layer's prompt on the card (57 → 123 tok/s since 28 Oct).



tooling/race7b.jsonl · first round of each, back to back at 94-95 °C (the second round hit the card's thermal floor)

A model too big for the card